

Modeling Like Peeling an Onion: Layerwise Analysis-Driven Automatic Behavioral Model Generation

Yike Huang*
Shanghai Key Laboratory of
Trustworthy Computing, East China
Normal University
Shanghai, Shanghai, China
huangyike@stu.ecnu.edu.cn

Ming Hu*
Shanghai Key Laboratory of
Trustworthy Computing, East China
Normal University
Shanghai, Shanghai, China
ecnu_hm@163.com

Xiaohong Chen†
Shanghai Key Laboratory of
Trustworthy Computing, East China
Normal University
Shanghai, Shanghai, China
xhchen@sei.ecnu.edu.cn

Zhi Jin†
Wuhan University
Wuhan, China
Peking University
Beijing, China
zhijin@whu.edu.cn

Shuyuan Xiao
East China Normal University
Shanghai, Shanghai, China
10235101446@stu.ecnu.edu.cn

Abstract

As software complexity skyrockets and requirements evolve at breakneck speed, traditional human-centric behavioral modeling can no longer keep pace in terms of efficiency, accuracy, and scalability. While existing automated approaches can produce models, they still struggle with deep semantic understanding of textual requirements or with reasoning about intricate system logic, especially nested relationships. Inspired by the way experienced analysts “peel back” layers of a problem, we propose LATO, a **Layerwise Analysis-Driven AuTomatic Behavioral MOdeling** approach. It employs a progressive decomposition strategy to guide large language models in incrementally parsing requirement structures, deconstructing behavioral dependencies, and ultimately generating executable UML activity diagrams. Comprehensive evaluations on four open-source datasets and two real-world industrial systems show that LATO comprehensively outperforms state-of-the-art baselines in accuracy, completeness, and syntactic compliance: F_1 scores for behavioral node-extraction improve by up to 71.1%, relation-extraction F_1 by 52.4% relatively, and syntactic pass rates remain above 96.67%. The framework also exhibits strong robustness to input perturbations, confirming its cross-domain generalizability. This paper is the first to tightly fuse human-inspired strategies with LLMs in behavior modeling, yielding an intelligent infrastructure that exhibits expert-level logical understanding and generalization. By closing the modeling-skills gap, LATO delivers a next-generation, low-cost, and explainable solution for requirements engineering and AI-native software development.

*Both authors are joint first authors.

†Both authors are co-corresponding authors.



This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License.

ICSE '26, Rio de Janeiro, Brazil

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2025-3/26/04

<https://doi.org/10.1145/3744916.3787806>

CCS Concepts

• **Software and its engineering** → **Requirements analysis.**

Keywords

Requirements Engineering, Behavioral Model Generation, Large Language Model, Layerwise Analysis

ACM Reference Format:

Yike Huang, Ming Hu, Xiaohong Chen, Zhi Jin, and Shuyuan Xiao. 2026. Modeling Like Peeling an Onion: Layerwise Analysis-Driven Automatic Behavioral Model Generation. In *2026 IEEE/ACM 48th International Conference on Software Engineering (ICSE '26)*, April 12–18, 2026, Rio de Janeiro, Brazil. ACM, New York, NY, USA, 13 pages. <https://doi.org/10.1145/3744916.3787806>

1 Introduction

Behavioral models (e.g., state machines, activity diagrams, and sequence diagrams) serve as a critical bridge between software requirements and implementation, significantly enhancing development efficiency. By formally capturing system dynamics through state transitions, activity flows, and interaction patterns, behavioral models provide an executable specification that aligns stakeholder expectations with technical realization [22]. This intermediate representation not only facilitates clearer communication across development teams, but also enables early validation through techniques like model checking [3, 4] and simulation [13] to identify design flaws, requirements inconsistencies, and edge cases during the initial development phases. Their applicability is broad: they are indispensable in safety-critical systems development [3], while in agile environments, they act as lightweight disambiguation tools [17].

Typically, the quality of behavioral models depends on the requirements engineer's expertise and experience in formal modeling [8, 36]. However, the construction of accurate and complete behavioral models faces growing challenges due to i) limited availability of requirements engineers with specialized expertise in formal modeling, and ii) the escalating scale and complexity of modern software systems. This dual constraint frequently results in manually derived behavioral models that either inadequately capture system requirements or contain errors that propagate through downstream development phases [12, 18]. In addition, the widespread adoption

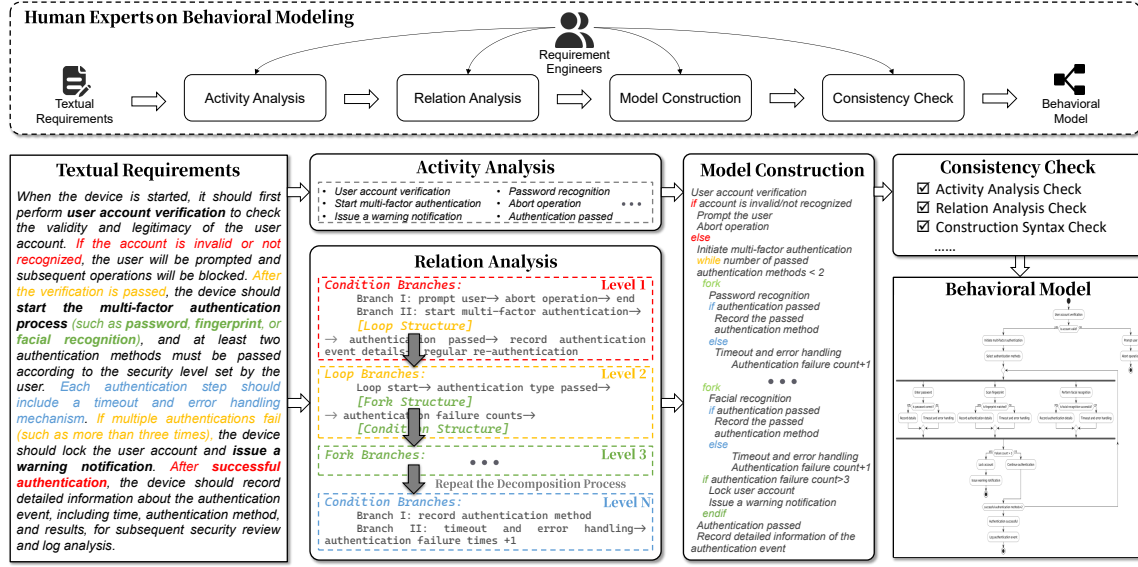


Figure 1: An Illustrative Workflow of Human Expert-Driven Behavioral Modeling.

of agile development methodologies and accelerated software release cycles has further exacerbated the limitations of manual behavioral modeling, rendering traditional approaches increasingly inadequate for maintaining model accuracy while meeting demands for rapid iteration and reliable system delivery [1, 20].

To achieve more efficient behavioral modeling, existing studies present automated generation methods to automatically generate behavioral models from textual requirements, which fall mainly into three categories, i.e., Rule-based methods [15, 24], Machine Learning (ML)-based methods [16, 40, 43], and Large Language Model (LLM)-based methods [17, 21, 37] respectively. Specifically, the rule-based methods primarily rely on predefined templates to (semi-)automatically extract key elements and generate corresponding behavioral models. ML-based methods [16, 40, 43] adopt various machine learning models to automatically learn patterns from a vast amount of software requirements data. Recent LLM-based approaches [17, 21, 25, 37] try to leverage LLMs' powerful natural language understanding and generation capabilities for modeling, free from the constraints of sentence structure and training data, thus showing great potential. Preliminary evaluations of these methods [9, 17] indicate that they perform well on straightforward tasks such as generating simple sequence diagrams.

However, existing LLM-based approaches still exhibit significant limitations when handling complex nested behavioral relationships, as they struggle to accurately capture multi-layered control logic (e.g., parallel branches within loops) in requirements. Consider the user account authentication flow illustrated in Fig. 1, a case that combines conditional, sequential, and concurrent operations. Existing approaches, lacking a proper grasp of its layered dependencies, produce models plagued by systematic deficiencies. Structurally, these include missing links and broken integrity; semantically, they involve redundant, incorrectly simplified, or omitted critical dependencies and timing sequences. This implies that the generated code, while potentially syntactically correct, suffers from compromised logical consistency, which, in turn, invalidates its executability in

practice. Consequently, there is an urgent need to develop automated modeling approaches capable of effectively handling complex nested behavioral relationships.

We note that even professional engineers struggle to directly extract complex behavioral relations from requirements and must rely on layered analysis strategies. Inspired by this, we propose LATO, a **Layerwise Analysis-Driven AuTomatic Behavioral MOdeling** approach. LATO constructs a novel LLM-powered framework whose core innovation is the systematic integration of expert-inspired layered reasoning with the semantic comprehension capabilities of LLMs. By emulating the progressive analytical process employed by humans, the framework can deconstruct hierarchical dependencies within requirements layer by layer, automatically extract and synthesize behavioral relations, and thereby accomplish the automated construction of behavioral models from complex requirements.

To evaluate the effectiveness of LATO, we conducted extensive experiments on four commonly used datasets and two real-world system scenario datasets that we collected. The results show that, compared with several state-of-the-art methods, LATO achieves the best performance in terms of extraction accuracy and robustness, especially in scenarios with deeply nested hierarchies. Concretely, it boosts behavioral-node extraction F_1 by up to 71.1% and relation-extraction F_1 by up to 52.4%, while maintaining a syntactic pass rate above 96.7%. The approach remains highly resilient to input perturbations, underscoring its cross-domain generalizability. This confirms its role as an intelligent infrastructure endowed with expert-level logical understanding and cross-domain generalization in behavior modeling. By effectively bridging the expertise gap in behavioral modeling, LATO delivers a next-generation solution that is efficient, low-cost, and explainable for requirements engineering and AI-native software development.

In summary, this paper makes the following contributions:

- We propose LATO, a layerwise analysis-driven automatic behavioral modeling approach. Our approach pioneers the systematic integration of hierarchical reasoning with LLM

workflows, effectively overcoming the limitations of existing methods in capturing complex behavioral logic.

- We evaluate LATO on four widely adopted open-source benchmark datasets as well as two scenario-specific datasets gathered from real-world industrial systems to comprehensively demonstrate the effectiveness.

Data Availability The related source codes and datasets in this paper are available <https://github.com/reg-repo/LATO>.

2 Expertise-inspired Framework Design

To equip LLMs with expert-level logical understanding and cross-domain generalization of behavioral modeling, the core idea of this paper is to distill a universal behavioral-modeling framework for LLMs from the modeling practices of human experts [32, 41]. As shown in Fig. 1, the core of expert methodology lies in the hierarchical decomposition strategy and systematic integration process: experts first deconstruct complex requirements into fundamental elements (activities and relations) through layered analysis, then systematically reassemble these elements into a coherent behavioral model (verified through consistency checking).

This cognitive paradigm directly shapes our three-component architecture: the **Key Activity Identifier** inherits the expert’s capability to rapidly pinpoint core entities in the initial phase; the **Layer-wise Relation Extractor** recreates the expert’s thinking mode in deconstructing complex logic—by establishing a hierarchical analytical framework to progressively unravel nested relationships; finally, the **Behavioral Model Constructor** adopts the expert’s systematic thinking during the integration phase, while internalizing the originally independent quality checks into self-verification mechanisms within each component.

Note that our approach goes beyond merely replicating the procedural steps of expert workflows—it internalizes their quality control logic. By implementing LLM-driven verification checkpoints at each component’s output node, we effectively transform the experts’ deliberative decision-making processes into executable validation steps. This design preserves the “divide and conquer” methodological advantage of human experts while achieving significant improvements in both quality and efficiency through the scalable reasoning capabilities of LLMs.

The resulting LATO framework (as shown in Fig. 2) essentially represents a structurally transformed expert workflow: it maintains the rationality of human cognitive processes while breaking through the efficiency limitations of manual operations via an LLM-tool collaborative paradigm, ultimately achieving effective integration of expert knowledge and LLM reasoning capabilities.

Guided by this design philosophy, LATO achieves automated modeling through the coordinated operation of three core modules: The **Key Activity Identifier** identifies core entities, employing LLMs to directly extract fundamental activity elements from text; The **Layer-wise Relation Extractor** deconstructs complex logic, adopting a hierarchical analysis strategy to progressively uncover relationships between elements; The **Behavioral Model Constructor** transforms structured elements into formal models while maintaining semantic consistency. Each module follows an “LLM Core Reasoning + External Tool-assisted” architecture, ensuring both

depth of semantic understanding and reliability of the modeling process. The detailed explanation of each module is as follows.

Key Activity Identifier: As depicted in Fig. 2, this module first employs the Activity Extractor to identify activities within the textual requirements. An external tool-based Activity Filter is then used to remove any spurious activities that do not appear in the original requirements. Finally, the LLM-based Activity Calibrator assesses the accuracy of the identified activities.

Layer-wise Relation Extractor: Due to the complex contexts typically involved in relationships, the layered relation extractor adopts an iterative approach to extract relations layer by layer based on the requirements structure. As shown in Fig. 2, the process operates as follows: First, the LLM-based framework extractor parses the layer framework of the inputted textual requirements. Then, the LLM-based layer populator performs detailed content population based on the extracted framework. Finally, the external tool-based layer inspector analyzes the descriptions generated in the previous steps to verify the relations within the current layer, and feeds the sub-layer descriptions back into the layer populator to initiate the next round of population.

Behavioral Model Constructor: This module initiates the process with the Structure Analyzer, which leverages elements extracted by the Key Activity Identifier and Layer-wise Relation Extractor to construct a sketch model structure. Subsequently, the Model Generator refines the sketch, culminating in the generation of a comprehensive behavioral model. To ensure correctness, the external tool-based Syntax Checker validates the generated model and generates the eventual artifacts, e.g., UML activity diagrams.

3 Methodology Implementation

3.1 Architecture Selection and Prompt Design

The system architecture and prompt design in this work adhere to systematic and modular engineering principles. Given that the core modules (activity identification → relation extraction → information integration) exhibit strict temporal dependencies and require no dynamic interaction, we adopt a pipeline architecture. Based on this architectural choice, we further design corresponding prompts, with the detailed design process described below.

1). **Modular Task Decomposition and Role Definition:** Each module in the pipeline is assigned a single, well-defined objective (e.g., activity identification). This highly focused design makes the construction of corresponding prompts relatively straightforward. Each prompt explicitly defines the role the LLM plays in a specific step, the task instructions it must follow, and the output constraints it must satisfy. For instance, the Activity Extractor, as the starting point of the pipeline, is strictly defined to identify atomic activities from the text without any additional commentary or reasoning.

2). **Iterative Optimization with Structured Templates:** We enhance the model’s capability to analyze complex requirements by integrating Chain-of-Thought (CoT) reasoning and few-shot examples into the prompt template. Each template comprises the following components: task objective, few-shot examples, operational instructions, format constraints, and input text specifications. For instance, the activity extractor template shown on the left side of Fig. 3 incorporates the instruction “do not include any additional commentary” to ensure the generation of a standardized activity list.

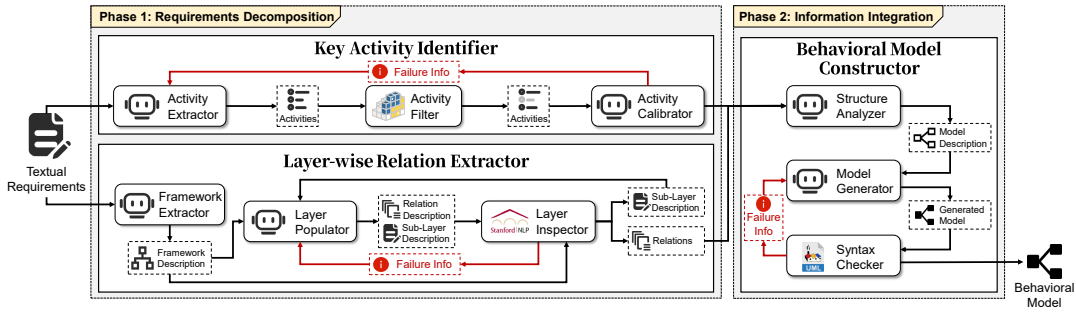


Figure 2: An Overview of LATO.

All templates underwent multiple rounds of iterative refinement on a held-out development set until performance stabilized.

Few-shot examples are selected based on dual criteria: diversity in requirement structures and comprehensive coverage of fundamental control-flow patterns (sequence, branch, and loop). Each example is structured as an input–output pair to explicitly demonstrate the task input and output. For instance, an input representing a sequential relation is: “As a curator, I need to standardize the unique identifiers of collection authors to easily view all their works together.” The corresponding output is a structured list: [“Identify authors”, “View all works”].

3). Finalization and Open-Source Release: All prompts were ultimately solidified into static templates to ensure consistent experimental conditions. The final prompt templates used in this study have been fully disclosed in the artifact repository: <https://github.com/reg-repo/LATO>, to ensure the reproducibility.

3.2 Key Activity Identifier

This module is dedicated to the precise extraction of behavioral activities from unstructured textual requirements, thereby establishing the essential atomic units that form the foundation for effective behavioral modeling. Its core operation relies on the collaborative work of three specialized components: the Activity Extractor, the Activity Filter, and the Activity Calibrator.

Activity Extractor: The Activity Extractor serves as the starting point of the extraction pipeline. It utilizes a carefully designed structured prompt template (Fig. 3) to systematically guide the LLM in activity identification by clearly defining task objectives, input text, and output format. The output is a standardized activity list based purely on textual features, with formats such as: “User Account verification, Abort Operation, ...”.

Activity Filter: This component is responsible for semantic purification. It employs the FastCoRef 2.1.6 [29] tool to resolve referential relationships within the text. This component can accurately establish associations between pronouns and their antecedents (e.g., mapping “it” to “the system”), effectively identifying and filtering out redundant behaviors caused by ambiguous references, thereby ensuring the clarity and relevance of the output activity set.

Activity Calibrator: This component enhances semantic consistency and textual grounding. During this phase, the LLM performs iterative refinement through a constrained multi-round loop, systematically comparing its extraction results with the original requirements. This calibration process (prompts are shown in Fig. 3)

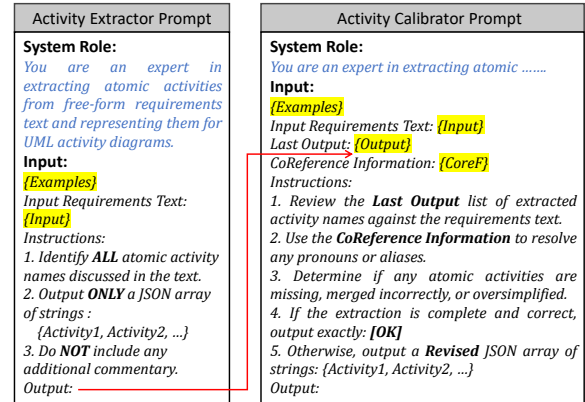


Figure 3: Prompt Templates for the Key Activity Identifier.

includes comprehensive validation of initial extractions, resolution of remaining coreference issues, and thorough completeness verification. The self-correcting workflow continues until the activity set achieves optimal semantic alignment with the source text, producing reliable atomic units for behavioral modeling.

Collaboration Process: The process begins with the Activity Extractor generating candidate activity sets, which are then semantically purified by the Activity Filter, and finally validated for quality by the Activity Calibrator. When quality validation fails, the system activates a targeted feedback mechanism: the Activity Calibrator directly triggers the Activity Extractor to perform re-extraction while automatically bypassing the coreference resolution step.

3.3 Layer-wise Relation Extractor

The core objective of this module is to decompose hierarchical behavioral relations within the text—such as conditions, loops, and parallel flows—layer by layer, akin to peeling an onion, ultimately constructing clearly structured and well-layered activity relations. The entire decomposition process is accomplished through the collaborative efforts of three core components: the Framework Extractor, the Layer Populator, and the Layer Inspector.

Framework Extractor: This component employs a recursive parsing method, synergistically combining structured task instructions with curated few-shot examples to derive a logical skeleton from input requirements. The process is as follows.

The parsing process begins with top-level instructions that require the model to perform two key operations: first, relation type identification, which recognizes the primary logical relations (e.g.,

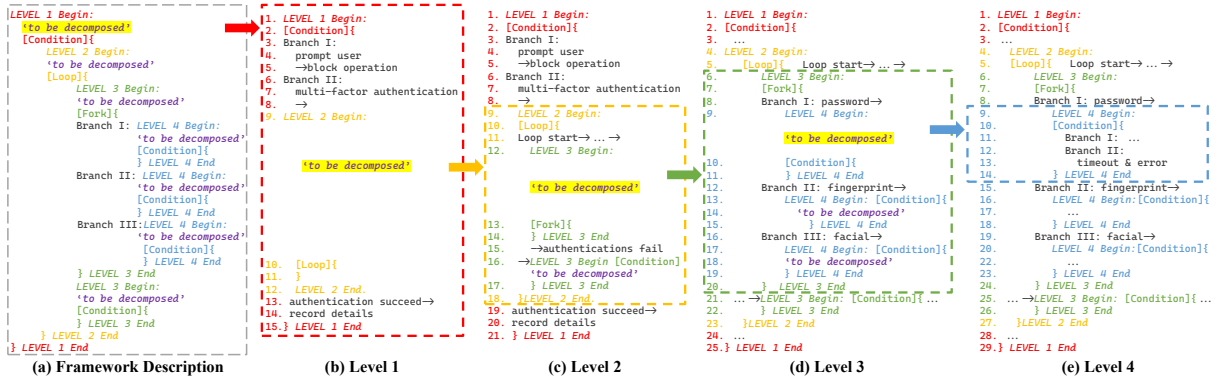


Figure 4: Illustration Result of An Example in Fig. 1.

sequence, condition, concurrency) within the current text layer; and second, hierarchical structure partitioning, which marks nodes with complex internal logic as “to be decomposed,” serving as entry points for recursive parsing. This partitioning relies on mapping rules established by few-shot examples, which define the correspondence between linguistic cues and standard structural relations: conditional relations are triggered by hypothetical phrases like “if-then”; sequential relations are indicated by temporal markers like “first-then”; concurrent relations are signaled by parallel indicators like “simultaneously”; and iteration patterns are characterized by repetitive phrases like “loop.”

When nodes marked “to be decomposed” appear in the model output, the extractor automatically triggers a recursive process. It creates a new parsing instance for each such node, injecting specialized lower-level instructions that inherit the parent context while clearly defining the parsing scope. This process iterates until all nodes are broken down into indivisible atomic units. Finally, the outputs from all levels are systematically integrated to construct a complete, well-structured logical framework.

For instance, when processing the input text shown in Fig. 1, the extractor generates the structured output presented in Fig. 4(a). This output reveals a clear four-layer logical architecture: the top-level (Level 1) decision drives the loop execution at the second level (Level 2); within the loop body, three concurrent processes (Level 3) and one conditional judgment (Level 3) are nested, with each concurrent process containing an internal conditional relation (Level 4). All levels maintain encapsulation of internal implementation details, representing core logical relationships solely through abstraction.

Layer Populator: This component is responsible for progressively populating the activity content for each substructure marked as “to be decomposed.” It employs an LLM-powered “onion-peeling” strategy to perform in-depth expansion of these marked nodes.

During execution, each substructure carrying the “to be decomposed” marker is processed recursively as an independent unit: the system identifies its internal logical patterns (such as conditional branches) and populates the corresponding specific activity steps. If newly generated nodes are still found to contain internal complexity during this process, they retain the “to be decomposed” marker, driving the parsing process to deeper levels. The recursive process automatically terminates when no more “to be decomposed” markers appear in the output, indicating that all substructures have been transformed into directly executable atomic-level activity units.

For example, taking the case shown in Fig. 4: at Level 1, Layer Populator identifies a conditional judgment with two branches (where Branch 2 contains the Level 2 structure requiring further parsing) and the sequential node “authentication succeeded,” with the final populated content depicted in lines 1-15 of Fig. 4(b). In the next iteration, the Populator further parses Level 2, identifying a loop structure along with its two internal Level 3 components and the “authentication failed” handling flow. The population results are shown in lines 9-18 of Fig. 4(c). In subsequent iterations, the parsing progresses to lower levels. For example, the Level 3 structure (lines 12-14 of Fig. 4(c)) is expanded into the detailed logic shown in lines 6-21 of Fig. 4(d). The Level 4 component within this logic (lines 9-12 of Fig. 4(d)) is which in turn refined into the implementation detailed in lines 9-12 of Fig. 4(e).

Layer Inspector: This component integrates Stanford CoreNLP 4.5.10 [31] to provide objective linguistic evidence to support semantic review based on LLMs. Leveraging the generated parse trees, the inspector verifies: the correctness of branch counts at the current layer, the accuracy of activity execution order within each branch, the completeness of identification and positioning for all nested substructures, and the consistency between the overall structure and both the input text semantics and the logic implied by the dependency tree. When parsing deviations are detected, the inspector automatically generates diagnostic reports containing specific linguistic evidence and sends these reports to the Layer Populator, providing precise guidance for subsequent targeted corrections.

Collaboration Process: The Framework Extractor first extracts the logical skeleton framework from the text and marks complex nodes as “to be decomposed”; the Layer Populator then receives this framework and recursively populates the marked nodes; the Layer Inspector subsequently validates the populated results—if issues are detected, it generates a diagnostic report and feeds it back to the Populator to trigger repopulation. This creates a closed-loop workflow of “extraction-population-inspection-repopulation,” which iterates until all nodes pass quality inspection, ensuring the accuracy and completeness of the parsing results.

3.4 Behavioral Model Constructor

In this module, we integrate the information extracted to generate an executable behavioral model. It includes three core components, Structural Analyzer, Model Generator, and Syntax Checker.

Structure Analyzer: This component is responsible for systematically integrating key activities and their hierarchical relationships extracted from previous phases. It aligns scattered activities and relationships into a unified structured intermediate representation, while preserving the full hierarchical semantics of the original requirements and organizing all components according to their logical relationships.

Model Generator: This component leverages LLMs to convert the intermediate output from the Structure Analyzer into formal diagram code (e.g., PlantUML [33]). The task of the generator is to map hierarchical activities and their relationships to syntactically correct modeling language constructs through the LLM.

Syntax Checker: This component validates the syntactic correctness of generated diagram. It utilizes the compilation feedback from the rendering engine (such as the PlantUML command-line tool (version v1.2025.4) [33]) as an authoritative basis for judgment to systematically detect syntax errors and structural defects in the code. Based on the detection results, the component generates detailed diagnostic reports that precisely identify error locations, types, and provide specific correction suggestions.

Collaboration Process: The Structure Analyzer passes the integrated structured intermediate representation to the Model Generator; the Model Generator then generates formal code and immediately submits it to the Syntax Checker for validation. The Checker obtains feedback through the PlantUML. When syntax errors are detected, it sends diagnostic reports containing specific error locations and correction suggestions back to the Generator, triggering a regeneration process. This “generate-check-regenerate” iteration loop continues until all diagram code passes the syntax check.

4 Experiment Design

To evaluate the effectiveness of LATO, we conducted a large-scale study to answer four research questions.

4.1 Research Questions

RQ1 (Performance Comparison) : How does LATO’s performance compare with existing generation methods? This question aims to evaluate the effectiveness of LATO. To address this RQ, we will conduct a comparative study by applying LATO and representative baseline methods to the same dataset using the same underlying LLM. Their performance will be evaluated using quality metrics to determine whether LATO achieves superior results.

RQ2 (Impact of selected input examples): How does the selection of example seeds influence the LATO’s ability to deal with complex logical requirements not seen in the examples? This question aims to investigate the generalizability of LATO. By examining outputs generated with diverse example seeds, this RQ seeks to uncover how seed selection shapes LATO’s effectiveness in addressing complex nested structures using the same LLM.

RQ3 (Ablation Study): What are the respective contributions of the three core components (Key Activity Identifier, Layer-wise Relation Extractor, and Behavioral Model Constructor) to the overall system performance? This question evaluates the contribution of each module. We selected one base LLM, and then individually removed a module and reported the fluctuations in result quality.

RQ4 (Different LLMs’ influence): What is the impact of different LLMs on the performance of LATO? This question investigates

the robustness of LATO. To this end, we selected several LLMs from different model families and evaluated their performance on the same dataset using LATO.

4.2 Datasets

The datasets selected for our experiments satisfy two key criteria: (i) **Diversity in complexity and domain:** The datasets span multiple domains and exhibit a range of requirement complexities, from simple to highly nested structures; (ii) **Representativeness of real-world requirements:** In particular, the constructed datasets are sourced from genuine industrial documents, ensuring practical relevance and authenticity. Experiments are conducted on six datasets, whose detailed statistics are presented in Table 1. The specifics of each dataset are described below.

- **Functional Scenario Descriptions (FSD)** [14] comprises 116 natural language descriptions from embedded systems, encompassing deeply nested functional behaviors such as device control, data processing, and fault handling.
- **Real Automotive Case (RAC)** [14] contains 20 requirement scenarios from the automotive domain, representing typical control logic and communication processes.
- **PURE Dataset (PURE)** [10] consists of 79 documents across multiple application domains and formats. For our evaluation, we extract 99 textual samples from simulated function-response sequences.
- **Business Process Dataset (BP)** [43] includes 30 examples of business software requirements, covering a range of operational scenarios.
- **User Stories Dataset (US)** [6] comprises 22 sets of user stories collected from various open-source projects, reflecting typical agile requirements.
- **LM Challenges (LMC)** [27] features 10 requirements documents from the cyber-physical systems domain.

Among these, the *FSD* and *RAC* datasets are newly constructed and collected directly from real-world industrial projects. The *FSD* dataset is sourced from authentic embedded system documentation and project artifacts, while the *RAC* dataset is obtained in collaboration with industry partners in the automotive sector. Both datasets contain original handwritten requirements that capture the complexity and specificity of actual engineering scenarios. Their inclusion ensures that our evaluation encompasses not only a broad spectrum of domains and complexity levels, but also the practical challenges encountered in industrial behavioral modeling tasks.

For all datasets, the corresponding activity diagrams were manually annotated using PlantUML to facilitate quantitative evaluation. The annotation was performed by three domain experts specializing in software engineering and behavioral modeling. All annotators adhered to a standardized guideline defining the identification of activities, relations, and control structures. Each expert independently labeled a shared subset of the data, and any discrepancies were resolved through discussion. The resulting Cohen’s kappa[5] exceeded 0.8 across all annotation dimensions, demonstrating high inter-annotator consistency and reliability.

Table 1: Statistics of the Datasets in Our Experiments

Dataset	#Scenario	#Nested Level	Domains
Functional Scenario Descriptions (FSD)	116	3-7(high-level)	Embedded System
Real Automotive Case (RAC)	20	2-6(high-level)	Automotive Industry
The PURE dataset (PURE)	99	1-3(simple)	Information System, Gaming, Networking...
Business Process dataset (BP)	30	1-3(simple)	Business Software
User Stories dataset (US)	220	1-2(simple)	Recycling System, Traffic Control...
Ten Lockheed Martin Cyber-Physical Challenges (LMC)	56	1-3(simple)	Cyber-physical System
In total	542	1-7	-

4.3 Evaluation Metrics

To rigorously assess the quality of the generated models, we employ a comprehensive set of metrics that evaluate both element-level fidelity and system-level validity. At the element level, we quantify the granular correctness of the generated models by computing *Precision*, *Recall*, and F_1 -score separately for two fundamental classes of elements: 1) Behavioral Nodes, representing atomic actions or states, and 2) Behavioral Relations, representing control flows or dependencies between nodes.

Precision measures the proportion of correctly generated elements among all produced elements. For n scenarios, let M_i denote the number of elements generated in scenario i and C_i the subset correctly matched to the ground truth. We report precision separately for nodes (*N-Precision*) and relations (*R-Precision*). The metric is defined as:

$$Precision = \frac{1}{n} \sum_{i=1}^n \frac{C_i}{M_i}. \quad (1)$$

Recall evaluates the completeness of the generated diagram in capturing all elements specified in the textual requirements. Let N_i denote the number of annotated ground-truth elements in scenario i . Node- and Relation-level *Recall* (*N-Recall* and *R-Recall*) are computed as:

$$Recall = \frac{1}{n} \sum_{i=1}^n \frac{C_i}{N_i}. \quad (2)$$

F_1 -score provides a balanced measure between *Precision* and *Recall* as follows:

$$F_1 = 2 \times \frac{Precision \times Recall}{Precision + Recall} \quad (3)$$

with $N-F_1$ and $R-F_1$ denoting the scores for node and relation extraction, respectively.

At the system level, **Pass Rate** [9, 17] is a metric that measures the proportion of generated models that are structurally valid and executable. Formally, *Pass Rate* is defined as

$$Pass Rate = \frac{k}{n} \quad (4)$$

where k is the number of valid models and n is the total number of generated models.

A model is considered valid only if it satisfies both *syntactic correctness* and *structural validity*. Syntactic correctness requires that the generated PlantUML code is fully parsable and compliant with the official specification. Structural validity encompasses *connectivity*, ensuring all nodes are reachable from a unique start node, and *integrity*, ensuring that all control flows have well-defined sources and targets with no dangling edges. Together, these criteria guarantee syntactically correct and structurally executable models.

Table 2: Budget and Average Latency of Different LLMs

Model	Input (\$ / 1M tokens)	Output (\$ / 1M tokens)	Avg. Response (s)
GPT-4	30.00	60.00	283
GPT-4.1	2.00	8.00	101
GPT-4o	2.50	10.00	94
Qwen3-8b	0.18	0.70	96
Qwen3-14b	0.35	1.40	228
Qwen3-32b	0.16	0.64	137
GLM-4-flash	0.00	0.00	175
GLM-4-air	0.07	0.28	87
GLM-4-plus	0.70	2.80	296
DeepSeek-V3	0.28	0.42	168

4.4 Comparison of Prompting Strategies

This paper presents a novel LLM-based workflow architecture for automated behavioral modeling. To assess its effectiveness, we benchmark it against the following mainstream LLM-based generation methods.

- *Zero-shot prompting* (0-shot) [19, 42] is directly to feed the textual requirements into LLMs and then obtain the generated model in response.
- *Few-shot prompting* (F-shot) [2] provides the LLMs with several example pairs of textual requirements and their corresponding models. LLMs then generate the model of input requirements based on the given examples.
- *Chain-of-Thought prompting* (CoT) [38] is a variation of few-shot prompting, adding extra thinking steps with concrete examples when feeding the LLMs.

For the comparative experiments based on LLMs, we constructed three types of baseline prompts following the prompt structures proposed in references [2, 19, 38, 42]. The few-shot examples used in RQ1, RQ3, and RQ4 comprise three samples randomly selected from a high-complexity dataset (FSD), chosen to ensure diversity in requirement structures and coverage of fundamental control-flow patterns (sequence, branch, and loop). Each example is presented as a complete input-output pair: the input consists of raw requirements text, while the output includes step-by-step reasoning followed by the structured model. These pairs are provided as contextual guidance prior to the target requirement in each prompt.

4.5 Base LLMs

To thoroughly evaluate the proposed workflow across diverse model characteristics, we selected ten LLMs spanning closed-source and

Table 3: Average Performance on Behavioral Node and Relation Extraction. Gain (%) denotes the percentage improvement of LATO over the best baseline for each metric.

Dataset	Metric	Node Extraction					Relation Extraction				
		0-shot	F-shot	CoT	LATO	Gain(%)	0-shot	F-shot	CoT	LATO	Gain(%)
FSD	<i>Precision</i>	0.6821	0.7540	0.7913	0.8654	+9.4	0.6606	0.6544	0.5651	0.7111	+7.6
	<i>Recall</i>	0.5932	0.7107	0.7481	0.8322	+11.2	0.3711	0.4124	0.4420	0.5296	+19.8
	<i>F₁</i>	0.6348	0.7318	0.7692	0.8484	+10.3	0.4752	0.5059	0.4960	0.6071	+20.0
RAC	<i>Precision</i>	0.6939	0.7500	0.6877	0.6905	−7.9	0.4419	0.4713	0.3777	0.5008	+6.3
	<i>Recall</i>	0.7858	0.7812	0.7904	0.8320	+5.3	0.3953	0.3853	0.3568	0.5142	+30.1
	<i>F₁</i>	0.7370	0.7653	0.7355	0.7547	−1.4	0.4173	0.4240	0.3669	0.5074	+19.7
PURE	<i>Precision</i>	0.7430	0.7925	0.8147	0.8891	+9.1	0.5095	0.5910	0.4568	0.6341	+7.3
	<i>Recall</i>	0.6654	0.7586	0.7852	0.8597	+9.5	0.4524	0.5107	0.5031	0.5570	+9.1
	<i>F₁</i>	0.7023	0.7752	0.7998	0.8742	+9.3	0.4793	0.5479	0.4788	0.5931	+8.3
BP	<i>Precision</i>	0.6538	0.6717	0.5341	0.6515	−3.0	0.4043	0.4586	0.3156	0.4523	−1.4
	<i>Recall</i>	0.3479	0.4008	0.4536	0.5180	+14.2	0.2129	0.2717	0.2661	0.3655	+34.5
	<i>F₁</i>	0.4542	0.5020	0.4906	0.5772	+15.0	0.2789	0.3412	0.2888	0.4043	+18.5
US	<i>Precision</i>	0.2210	0.2541	0.1214	0.5265	+107.2	0.2914	0.3186	0.1629	0.5969	+87.4
	<i>Recall</i>	0.6498	0.6756	0.6327	0.7895	+16.9	0.9700	0.9543	0.9180	0.9329	−3.8
	<i>F₁</i>	0.3299	0.3693	0.2037	0.6317	+71.1	0.4482	0.4777	0.2802	0.7280	+52.4
LMC	<i>Precision</i>	0.5095	0.5910	0.4568	0.6341	+7.3	0.8093	0.8574	0.7474	0.8598	+0.3
	<i>Recall</i>	0.4524	0.5107	0.5031	0.5570	+9.1	0.6830	0.7108	0.8048	0.7293	−9.4
	<i>F₁</i>	0.4793	0.5479	0.4788	0.5931	+8.3	0.7408	0.7773	0.7750	0.7892	+1.5

open-source, varying parameter scales, and multilingual capabilities, thereby ensuring broad coverage across major model categories and architectures. Their detailed specifications are provided below.

DeepSeek [23] is an advanced AI assistant developed by DeepSeek, which represents a high-quality open-source baseline. We utilized its official APIs to conduct the experiments.

GPT-4 [28] provides the state-of-the-art commercial upper bound, which offers a 128 K-token context window. We evaluated its specific variants GPT-4, GPT-4o and GPT-4. 1 via OpenAI’s API.

Qwen3 [39], introduces architectural diversity with dense and MoE variants in multiple sizes(e.g., 8B to 32B), and enables mode switching for reasoning and chat.

GLM-4 [11] contributes additional diversity from a different research group. In this paper, we utilized GLM-4-flash, GLM-4-air, and GLM-4-plus as the versions of base models.

The budget and average latency for each LLM were measured in a single execution and are reported in Table 2. To ensure experimental fairness and comparability, all baseline methods and LATO employed a unified sampling parameter configuration: the *temperature* was set to 0.75, *top_p* to 1, and the maximum completion tokens were adjusted to obtain outputs of sufficient length.

We selected DeepSeek-V3 as the base model for RQs 1, 2, and 3 for three key reasons: as an open-source model, it ensures reproducibility while avoiding versioning and cost concerns of commercial APIs; its competitive performance in RQ4 validates its suitability as a benchmark; and maintaining a consistent LLM isolates the effect of our methodological contributions in these experiments.

5 Results and Analysis

We investigate the above four research questions.

Table 4: Average Pass Rate for Each Method

Dataset	Zero-shot	Few-shot	CoT	LATO
FSD	0.8261	0.9429	0.9321	0.9679
RAC	0.9672	0.9545	0.9560	0.9708
PURE	0.9464	0.9757	0.9479	0.9764
BP	0.9730	0.8964	0.9864	0.9667
US	0.9814	0.9506	0.9257	0.9914
LMC	0.9879	0.9750	0.9743	0.9936

5.1 RQ1: Performance Comparison

In RQ1, we evaluate LATO and baselines across various datasets, calculating distinct performance scores to assess generation quality under diverse requirement scenarios.

Setup. We utilize the deepSeek-v3 model to apply both baselines and LATO. Semantic similarity is computed using embeddings produced by the all-MiniLM-L6-v2 model. A threshold of 0.85 is chosen through grid search in the range [0.50,0.95] [30], which yields a trade-off between metrics for elements matching.

Results. Tables 3 and 4 present a detailed comparative analysis of *Precision*, *Recall*, and *F₁* for behavioral node extraction and relation extraction, and overall *Pass Rate* on each dataset. Notably, LATO achieves notable improvements over the baselines, with the maximum *F₁* score increase reaching 71.1% for behavioral node extraction and 52.4% for behavioral relation extraction. On average, LATO outperforms the baselines by 19.0% in node and 20.0% in relation extraction *F₁* scores. Furthermore, LATO maintains a high *Pass Rate* of at least 96% across all datasets, suggesting its effectiveness and reliability in various scenarios.

Analyses. The examination of performance scores unveils several noteworthy insights into the capabilities of LATO in comparison with baselines. Beginning with node extraction, as detailed

Table 5: The Performance Scores of LATO with Different Example Seeds

Seed	N-Precision	N-Recall	N-F ₁		R-Precision	R-Recall	R-F ₁		Pass Rate	
Seed A	0.6905	0.8320	0.7547	–	0.5008	0.5142	0.5074	–	0.9708	–
Seed B	0.6840	0.8255	0.7492	0.73% ↓	0.4821	0.5010	0.4898	3.47% ↓	0.9676	0.33% ↓
Seed C	0.6702	0.8123	0.7350	2.61% ↓	0.4955	0.5108	0.5035	0.77% ↓	0.9690	0.18% ↓

Table 6: The Results of Ablation Study

	N-Precision	N-Recall	N-F ₁		R-Precision	R-Recall	R-F ₁		Pass Rate	
LATO	0.6905	0.8320	0.7547	–	0.5008	0.5142	0.5074	–	0.9708	–
w/o Identifier	0.6356	0.7257	0.6777	10.20% ↓	0.4178	0.4003	0.4089	19.41% ↓	0.9614	0.97% ↓
w/o Extractor	0.6232	0.7288	0.6719	10.97% ↓	0.3919	0.3886	0.3902	23.10% ↓	0.9592	1.19% ↓
w/o Constructor	0.6019	0.6687	0.6336	16.05% ↓	0.3519	0.3283	0.3397	33.06% ↓	0.9316	4.04% ↓

in Table 3, LATO achieves *Precision*, *Recall*, and *F₁*-score on high-complexity datasets (e.g., *FSD*) that are comparable to its performance on low-complexity datasets. It is particularly noteworthy that the lowest scores occur on simpler datasets (e.g., *US* and *LMC*). These results indicate that LATO demonstrates modeling stability in complex requirement scenarios compared to simple ones, with its performance being less affected by requirement complexity.

For relation extraction (summarized in Table 3), LATO’s performance shows a notable contrast: on complex datasets, it achieves up to a 20% improvement in *F₁*-score while maintaining a minimum precision baseline of 6.3%; whereas on simpler datasets, it reaches a maximum precision of 87.4% but may experience performance degradation of up to 9.4%. This comparative result demonstrates that LATO exhibits stronger robustness and adaptability when handling complex relational structures.

As shown in Table 4, the *Pass Rate* results demonstrate that while baseline methods perform acceptably on low-complexity datasets, their performance significantly deteriorates in complex scenarios (e.g., only 82.61% on the *FSD* dataset). In contrast, LATO consistently maintains performance (with a minimum *Pass Rate* of 96.67%). This clearly proves that LATO possesses stronger generalization capability and stability, effectively adapting to requirement scenarios of varying complexity, while particularly demonstrating significant advantages when handling complex logical structures.

Answer to RQ1: LATO shows overall better performance than baseline methods in both element extraction and generation success rate (at least 96.67%) across diverse datasets, indicating improved accuracy and completeness.

5.2 RQ2: Impact of Selected Input Examples

In this experiment, we explore the generalization capacity of LATO when faced with input seeds (samples) characterized by significantly varying levels of text complexity.

Setup. We assess LATO using input seeds that exhibit both high and low complexity levels, determined by the degree of relation nesting. Seed A employs the same input samples as the prior experiment, comprising three samples randomly selected from a high-complexity dataset, such as *FSD*. Seeds B and C consist of three samples each, randomly chosen from both high-level and simple complexity datasets. Standard evaluation metrics are utilized to measure the performance of each seed on the *RAC* dataset.

Results. The performance of LATO shows only minor variations with respect to input complexity, as detailed in Table 5. The N-F₁ metric experienced a decline of up to 2.61%, and R-F₁ fluctuated within a 3.47% range. Despite these variations, the *Pass Rate* remained approximately 0.97 across all scenarios, with no statistically meaningful decline observed.

Analyses. Table 5 demonstrates that the performance of LATO empirically shows low sensitivity to input complexity. The slight variations in the N-F₁ and R-F₁ metrics (decreasing by 2.61% and 3.47%, respectively) suggest that LATO maintains stable performance in analyzing and modeling input data, despite minor fluctuations associated with input complexity.

Importantly, the *Pass Rate* remains robust at approximately 0.97 across all experimental configurations, indicating that LATO tends to produce reliable results across varying input complexities. This stability indicates that even when the few-shot information introduced by input requirements fluctuates, LATO effectively maintains its performance, supported by self-mechanism features and tool support within the comprehensive framework design.

Answer to RQ2: LATO generalization capability when handling diverse and complex inputs, as evidenced by its ability to consistently maintain a high *Pass Rate* around 0.97 across different scenarios.

5.3 RQ3: Ablation Study

We systematically measure each module’s contribution to the overall framework’s performance.

Setup. Based on the established base model, we perform ablation studies by removing each major module from the LATO pipeline in turn and evaluating the resulting performance changes.

Results. Table 6 presents the effects of removing individual modules from the LATO pipeline, showing overall performance declines across all ablated variants compared to the complete model. Notably, removing the Activity Identifier or Relation Extractor results in a decrease of 10.20% and 10.97% in N-F₁ score, and a reduction of 19.41% and 23.10% in R-F₁ score, respectively. The largest observed performance drop is observed when the Constructor is omitted, leading to a 16.05% decrease in N-F₁, a 33.06% decline in R-F₁, and a 4.04% reduction in *Pass Rate*.

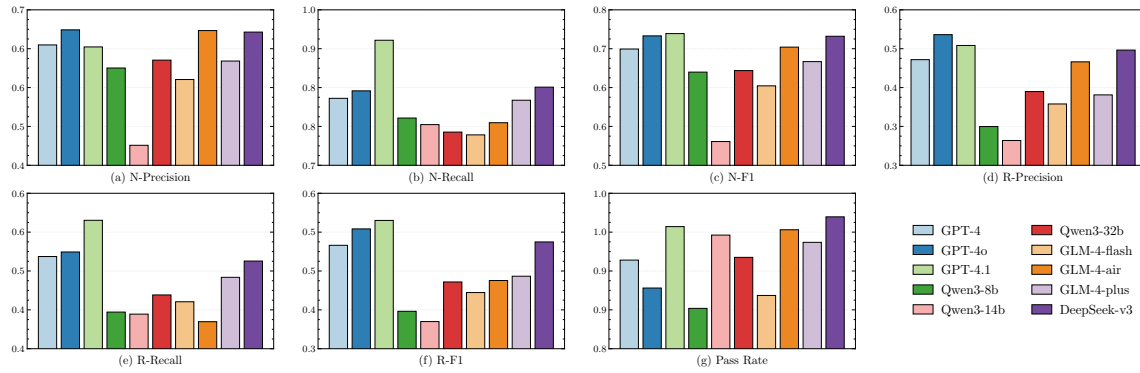


Figure 5: Performance Comparison of LATO across Different Base Models. The figure reports three metric groups: (1) Node Metrics – N-Precision, N-Recall, and N-F₁ score; (2) Relation Metrics – R-Precision, R-Recall, and R-F₁ score; and (3) Pass Rate.

Analyses. The ablation study results in Table 6 offer valuable insights into the roles of individual modules within the LATO framework. Removing key modules like the Identifier, Extractor, and Constructor leads to noticeable performance declines, highlighting their substantial contributions to overall system performance.

Omitting the Activity Identifier significantly impacts *Recall*, highlighting its important role in node identification and maintaining data retrieval accuracy. Similarly, the Relation Extractor’s removal notably reduces both *Precision* and *Recall*, indicating its key role in breaking down complex inputs for processing.

The absence of the Constructor results in the largest performance decline, particularly in *Recall* and *Pass Rate*. This indicates that integrating extracted information is an important factor in achieving comprehensive and accurate outputs.

These findings highlight the contribution of each module and suggest that all components play complementary roles within LATO’s architecture. The study stresses the importance of a cohesive system where each part enhances the others, ensuring robust performance and reliability across various applications.

Answer to RQ3: Each module of LATO contributes meaningfully to overall performance, with the Constructor showing the strongest influence on *Recall* and *Pass Rate*.

5.4 RQ4: Different LLMs’ Influence

RQ4 examines the sensitivity of LATO to varying LLM capabilities.

Setup. We utilized LATO by employing a variety of LLMs, including the GPT-4 series, Qwen3 models, and GLM-4 series. These models collectively represent different scales and configurations.

Results. As shown in Fig. 5, more advanced LLMs generally yield better results overall, though the improvements vary across performance metrics. Specifically, GPT-4.1 achieves the highest N-Recall (90.91%) and Pass Rate (95.87%), highlighting its relative strength in information extraction. However, its N-F₁ score displays only a modest edge over GPT-4o. Models such as Qwen3-32b and GLM-4-air show competitive node-level performance but comparatively weaker results in relation extraction. Conversely, smaller or older models like Qwen3-14b and GLM-4-flash exhibit noticeable declines in *Precision* and *F₁*, particularly in relation extraction.

Analyses. The examination of various LLMs within the LATO framework highlights the differential impact these models have on performance outcomes. Advanced models, particularly those

in the GPT-4 series, tend to achieve higher *Recall* and *Pass Rate*. Specifically, GPT-4.1 achieves the highest N-Recall and Pass Rate, underscoring its adeptness at capturing comprehensive information and enabling robust data processing suitable for complex tasks.

Models in the Qwen3 series, like Qwen3-32b, show acceptable performance on node-related metrics, indicating optimization for precision in identification. However, they require further enhancements in relation extraction tasks. Performance variations indicate that model capability exerts nuanced effects on different subtasks, pointing to potential directions for improvement.

The GLM-4 series, with GLM-4-air leading in *Pass Rate* and N-Precision, exhibits relatively stable and reliable outputs across metrics. Nonetheless, gaps in relational precision highlight opportunities to further refine its node identification capabilities.

DeepSeek-V3 demonstrates overall strong performance, with its node extraction, relation extraction, and *Pass Rate* metrics all ranking within the top 2-3 positions, achieving the highest Pass Rate among all models. The performance variations across different models indicate that our method does require a certain level of capability from the base LLMs, yet beyond a certain threshold, the improvement in final outcomes tends to plateau.

Answer to RQ4: LATO’s performance scales tightly with the underlying LLM’s capability: top-tier models like GPT-4 achieve the highest *Recall* and *Pass Rate* among compared models, whereas other models retain strength in node metrics but diverge markedly in relation extraction.

6 Threats to Validity

Construct Validity. ① *The Accuracy-Efficiency Trade-off:* Evaluation shows that while LATO achieves higher accuracy than single-pass baselines, its iterative design demands significantly more computational resources (e.g., API calls and latency). This establishes a new state-of-the-art in parsing fidelity but may limit its practicality in resource-constrained scenarios. Therefore, claims about LATO’s “effectiveness” should be contextualized: it excels in accuracy-critical tasks but incurs inherent efficiency costs.

② *Context Length Constraints:* Although excessively long requirement texts may reach the LLM’s context limit, a requirements document is typically divided into multiple independent behavioral

models, each with a modest description size. Consequently, current mainstream LLMs can readily accommodate the input.

③ *Domain Knowledge Limitation*: Behavioral modeling inherently hinges on the domain context. Our current approach targets general scenarios and remains domain-agnostic. For highly specialized domains, we recommend augmenting the LLM with domain semantics—via retrieval-augmented generation (RAG) or knowledge embedding—to further enhance performance.

④ *Selected Input Examples*: LATO relies on few-shot prompting; domain-specific instances could further boost performance. Nevertheless, the RQ2 experiments indicate that even generic examples yield only marginal impact, preserving overall robustness.

External Validity: ① *Scenario Coverage Breadth*: While the current dataset spans multiple domains, it contains limited industrial cases with relatively simple nested and concurrent requirements. The approach remains untested on ultra-large-scale, highly-coupled, or safety-critical systems. Future work will incorporate high-complexity industrial scenarios to rigorously evaluate LATO's robustness and generalizability under extreme requirements.

② *Evaluation Granularity*: Current evaluation focuses on LATO's overall effectiveness, but lacks fine-grained analysis by relation type or nesting pattern, leaving its consistency in specific scenarios (e.g., “parallel-within-loop” structures) unconfirmed. Aggregate metrics may mask localized performance fluctuations. Future work will develop fine-grained assessment profiles to more precisely characterize LATO's strengths, limitations, and applicable boundaries.

Conclusion Validity. The conclusions of our study may be affected by experimental design and data annotation. Although standardized annotation protocols and multiple experimental runs were employed to reduce random fluctuations, the use of human-annotated ground truth may still introduce subjective bias and inconsistency. Furthermore, the inherent stochasticity of LLM-based methods can lead to minor performance variations across trials. To mitigate these effects, we averaged results over multiple runs, though subtle sources of error may still remain.

7 Related Work

Rule-based Methods rely on predefined structured rules to automatically convert textual requirement descriptions into behavioral models. For example, Sholih *et al.* [34] extract semantic fact types and map them to BPMN constructs. Jahan *et al.* [16] employ rule-based parsing to generate UML sequence diagrams from natural-language requirement statements. De Biase *et al.* [7] automatically extract event, guard, and action patterns from Given-When-Then requirements and map them to predefined SysML transition templates. Despite their strengths, these methods often rely heavily on precise and well-defined inputs, which may not always be available.

ML-based Methods learn the structure and rules of behavioral models by analyzing system data or manually annotated information. For example, Zhu *et al.* [43] employ richly annotated pairs of business text and UML activity diagrams to deeply supervise a neural generator for modeling. Mammadov *et al.* [26] introduce Modelizer, a seq2seq model that learns reversible behavior from synthetic input-output pairs. Tavassoli *et al.* [35] propose PL*, an adaptive L*-based learning method that reuses sequences across

product variants. Although such methods can capture system behavioral characteristics, their performance is highly dependent on the scale and quality of training data.

LLM-based Modeling Methods leverage the powerful semantic understanding and generation capabilities of LLMs to automatically convert natural language requirements into behavioral models. For instance, Kourani *et al.* iteratively prompt LLMs to generate and refine BPMN/Petri-net models with quality controls [21]. Jahan *et al.* conducted a comparative study exploring the capabilities of generative AI for generating UML sequence diagrams from user stories [17]. Wang *et al.* demonstrated that LLMs can assist novice analysts in generating sequence diagrams [37]. Ferrari *et al.* systematically evaluated ChatGPT's ability to create UML sequence diagrams from requirements [9], noting its fluent syntax but occasional content omissions, highlighting the potential for LLMs to streamline modeling tasks. However, none of these methods provides a systematic solution for behavioral modeling with complex nested relationships.

In contrast, this paper employs a layer-wise relation decomposition strategy to guide LLMs in relation extraction, thereby more accurately handling complex behavioral logic. To the best of our knowledge, our work presents the first method that decomposes complex nested behavioral logic for guided LLM-based modeling via an expertise-inspired workflow.

8 Conclusion

This paper proposes LATO, a layerwise analysis-driven automatic behavioral modeling approach specifically designed for handling complex behavioral logic. It employs a layered reasoning mechanism that replicates how domain experts process complex requirements, guiding LLMs through a progressive analysis of unstructured textual inputs: starting with deconstructing fundamental behavioral elements, then clarifying nested relationships and constraints among these components, and ultimately constructing executable models with complete logical semantics. Experiments on cross-domain benchmarks demonstrate that LATO significantly outperforms existing methods in the modeling accuracy of complex behavioral relationships while maintaining stable generalization capabilities across diverse domains and model architectures.

This demonstrates that the combination of “human-inspired strategies + LLM” not only enables more intelligent and natural understanding of the implicit logic in complex requirement texts, akin to human comprehension, but also offers scalable and cost-effective adaptability to complex scenarios. This approach provides a new infrastructure-level capability for requirements engineering, system design, and even AI-native software development.

Looking ahead, we plan to extend LATO to support a broader range of output formats and domain-specific modeling languages, thereby enhancing its applicability across diverse industrial contexts. Simultaneously, we will delve into exploring the accuracy-efficiency Pareto frontier, establishing differentiated resource allocation strategies for high-responsiveness scenarios like real-time control systems and high-accuracy scenarios such as safety-critical systems, to better balance performance and resource consumption. Finally, as LLMs continue to evolve, we aim to investigate

the integration of more advanced architectures to further boost performance, interpretability, and efficiency.

Acknowledgments

This work is supported by the National Natural Science Foundation of China under Grant No. 62192731, 92582203, 62272166, and 62192730.

References

- [1] Sharif Ahmed, Arif Ahmed, and Nasir U. Eisty. 2022. Automatic Transformation of Natural to Unified Modeling Language: A Systematic Review. In *2022 IEEE/ACIS 20th International Conference on Software Engineering Research, Management and Applications (SERA)*. 112–119. doi:10.1109/SERA54885.2022.9806783
- [2] Tom Brown, Benjamin Mann, Nick Ryder, and et al. 2020. Language Models are Few-Shot Learners. In *Advances in Neural Information Processing Systems*, H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, and H. Lin (Eds.), Vol. 33. Curran Associates, Inc., 1877–1901. https://proceedings.neurips.cc/paper_files/paper/2020/file/1457c0d6bfc4967418bfb8ac142f64a-Paper.pdf
- [3] Xiaohong Chen, Qianqian Liu, Frédéric Mallet, Qin Li, Shubin Cai, and Zhi Jin. 2022. Formally verifying consistency of sequence diagrams for safety critical systems. *Science of computer programming* 216 (2022), 102777.
- [4] Xiaohong Chen, Frédéric Mallet, and Xiaoshan Liu. 2020. Formally verifying sequence diagrams for safety critical systems. In *2020 International Symposium on Theoretical Aspects of Software Engineering (TASE)*. IEEE, 217–224.
- [5] Jacob Cohen. 1960. A coefficient of agreement for nominal scales. *Educational and psychological measurement* 20, 1 (1960), 37–46.
- [6] Fabiano Dalpiaz and Arnon Sturm. 2020. Conceptualizing Requirements Using User Stories and Use Cases: A Controlled Experiment. In *Requirements Engineering: Foundation for Software Quality: 26th International Working Conference, REFSQ 2020, Pisa, Italy, March 24–27, 2020, Proceedings* (Pisa, Italy). Springer-Verlag, Berlin, Heidelberg, 221–238. doi:10.1007/978-3-030-44429-7_16
- [7] Maria Stella de Biase, Simona Bernardi, Stefano Marrone, José Merseguer, and Angelo Palladino. 2024. Completion of SysML state machines from Given–When–Then requirements. *Software and Systems Modeling* 23, 6 (2024), 1455–1491.
- [8] Michel dos Santos Soares, Jos Vrancken, and Alexander Verbrack. 2011. User requirements modeling and analysis of software-intensive systems. *Journal of Systems and Software* 84, 2 (2011), 328–339.
- [9] Alessio Ferrari, Sallam Abualhajja, and Chetan Arora. 2024. Model Generation with LLMs: From Requirements to UML Sequence Diagrams. In *2024 IEEE 32nd International Requirements Engineering Conference Workshops (REW)*. 291–300. doi:10.1109/REW61692.2024.00044
- [10] Alessio Ferrari, Giorgio Oronzo Spagnolo, and Stefania Gnesi. 2017. PURE: A Dataset of Public Requirements Documents. In *2017 IEEE 25th International Requirements Engineering Conference (RE)*. 502–505. doi:10.1109/RE.2017.29
- [11] Team GLM, Aohan Zeng, Bin Xu, Bowen Wang, Chenhui Zhang, Da Yin, Dan Zhang, Diego Rojas, et al. 2024. Chatglm: A family of large language models from glm-130b to glm-4 all tools. *arXiv preprint arXiv:2406.12793* (2024).
- [12] Bogumiła Hnatkowska and Mateusz Cebinka. 2021. Activity Diagram Generation Based on Use-Case Textual Specification. *Computing and Informatics* 40, 4 (Dec. 2021), 772–795. doi:10.31577/cai_2021_4_772
- [13] Ming Hu, Wenxue Duan, Min Zhang, Tongquan Wei, and Mingsong Chen. 2020. Quantitative timing analysis for cyber-physical systems using uncertainty-aware scenario-based specifications. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 39, 11 (2020), 4006–4017.
- [14] Yike Huang. 2025. LATO. <https://github.com/reg-repo/LATO>
- [15] Ana Ivanchikj, Souhaila Serbout, and Cesare Pautasso. 2020. From text to visual BPMN process models: design and evaluation. In *Proceedings of the 23rd ACM/IEEE International Conference on Model Driven Engineering Languages and Systems (Virtual Event, Canada) (MODELS '20)*. Association for Computing Machinery, New York, NY, USA, 229–239. doi:10.1145/3365438.3410990
- [16] Munima Jahan, Zahra Shakeri Hossein Abad, and Behrouz Far. 2021. Generating sequence diagram from natural language requirements. In *2021 IEEE 29th International Requirements Engineering Conference Workshops (REW)*. IEEE, 39–48.
- [17] Munima Jahan, Mohammad Mahdi Hassan, Reza Golpayegani, Golshid Ranjbaran, Chanchal Roy, Banani Roy, and Kevin Schneider. 2024. Automated Derivation of UML Sequence Diagrams from User Stories: Unleashing the Power of Generative AI vs. a Rule-Based Approach. In *Proceedings of the ACM/IEEE 27th International Conference on Model Driven Engineering Languages and Systems*. 138–148.
- [18] Muhammad Javed and Yuqing Lin. 2021. iMER: Iterative process of entity relationship and business process model extraction from the requirements. *Information and Software Technology* 135 (2021), 106558.
- [19] Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. 2022. Large language models are zero-shot reasoners. *Advances in neural information processing systems* 35 (2022), 22199–22213.
- [20] Shekoufeh Kolehdoz-Rahimi, Kevin Lano, and Chenghua Lin. 2023. Requirement formalisation using natural language processing and machine learning: A systematic review. *arXiv preprint arXiv:2303.13365* (2023).
- [21] Humam Kourani, Alessandro Berti, Daniel Schuster, and Wil MP van der Aalst. 2024. Process modeling with large language models. In *International Conference on Business Process Modeling, Development and Support*. Springer, 229–244.
- [22] Hatice Koç, Ali Mert Erdoğan, Yousef Barjakly, and Serhat Peker. 2021. UML Diagrams in Software Engineering Research: A Systematic Literature Review. *Proceedings* 74, 1 (2021). doi:10.3390/proceedings2021074013
- [23] Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. 2024. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437* (2024).
- [24] Abdelsalam M. Maatuk and Esra A. Abdelnabi. 2021. Generating UML Use Case and Activity Diagrams Using NLP Techniques and Heuristics Rules. In *International Conference on Data Science, E-Learning and Information Systems 2021 (Ma'an, Jordan) (DATA'21)*. Association for Computing Machinery, New York, NY, USA, 271–277. doi:10.1145/3460620.3460768
- [25] Zhi Ma, Cheng Wen, Zhexin Su, Xiao Liang, Cong Tian, Shengchao Qin, and Mengfei Yang. 2025. Bridging Natural Language and Formal Specification - Automated Translation of Software Requirements to LTL via Hierarchical Semantics Decomposition Using LLMs. In *Proceedings of the 40th IEEE/ACM International Conference on Automated Software Engineering*.
- [26] Tural Mammadov, Dietrich Klakow, Alexander Koller, and Andreas Zeller. 2025. Learning Program Behavioral Models from Synthesized Input-Output Pairs. *ACM Trans. Softw. Eng. Methodol.* (July 2025). doi:10.1145/3748720
- [27] Anastasia Mavridou, Hamza Bourboui, Dimitra Giannakopoulou, Thomas Pressburger, Mohammad Hejase, Pierre-Loic Garoche, and Johann Schumann. 2020. The Ten Lockheed Martin Cyber-Physical Challenges: Formalized, Analyzed, and Explained. In *2020 IEEE 28th International Requirements Engineering Conference (RE)*. 300–310. doi:10.1109/RE48521.2020.00040
- [28] OpenAI. 2024. GPT-4o: OpenAI's Multimodal AI Model. <https://openai.com>
- [29] Shon Otmazgin, Arie Cattan, and Yoav Goldberg. 2022. F-coref: Fast, accurate and easy to use coreference resolution. *arXiv preprint arXiv:2209.04280* (2022).
- [30] David M. W. Powers. 2020. Evaluation: from precision, recall and F-measure to ROC, informedness, markedness and correlation. *arXiv:2010.16061 [cs.LG]* <https://arxiv.org/abs/2010.16061>
- [31] Peng Qi, Timothy Dozat, Yuhao Zhang, and Christopher D. Manning. 2018. Universal Dependency Parsing from Scratch. In *Proceedings of the CoNLL 2018 Shared Task: Multilingual Parsing from Raw Text to Universal Dependencies*, Daniel Zeman and Jan Hajič (Eds.). Association for Computational Linguistics, Brussels, Belgium, 160–170. doi:10.18653/v1/K18-2016
- [32] William E. Riddle, Jack C. Wileden, John H. Saylor, Alan R. Segal, and Allan M. Stavely. 1978. Behavior modelling during software design. In *Proceedings of the 3rd International Conference on Software Engineering* (Atlanta, Georgia, USA) (ICSE '78). IEEE Press, 13–22.
- [33] Arnaud Roques and PlantUML Contributors. 2025. *PlantUML Software*. PlantUML Team. <https://github.com/plantuml/plantuml>
- [34] Sholiq Sholiq, Rianarto Sarno, and Endang Siti Astuti. 2022. Generating BPMN diagram from textual requirements. *Journal of King Saud University-Computer and Information Sciences* 34, 10 (2022), 10079–10093.
- [35] Shaghayegh Tavassoli, Carlos Diego N Damasceno, Ramtin Khosravi, and Mohammad Reza Mousavi. 2022. Adaptive behavioral model learning for software product lines. In *Proceedings of the 26th ACM International Systems and Software Product Line Conference-Volume A*. 142–153.
- [36] Sebastian Uchitel, Jeff Kramer, and Jeff Magee. 2003. Synthesis of behavioral models from scenarios. *IEEE Transactions on Software Engineering* 29, 2 (2003), 99–115. doi:10.1109/TSE.2003.1178048
- [37] Beian Wang, Chong Wang, Peng Liang, Bing Li, and Cheng Zeng. 2024. How LLMs Aid in UML Modeling: An Exploratory Study with Novice Analysts. In *2024 IEEE International Conference on Software Services Engineering (SSE)*. 249–257. doi:10.1109/SSE62657.2024.00046
- [38] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. 2022. Chain-of-thought prompting elicits reasoning in large language models. In *Proceedings of the 36th International Conference on Neural Information Processing Systems* (New Orleans, LA, USA) (NIPS '22). Curran Associates Inc., Red Hook, NY, USA, Article 1800, 14 pages.
- [39] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388* (2025).
- [40] Yanming Yang, Xin Xia, David Lo, and John Grundy. 2022. A Survey on Deep Learning for Software Engineering. *ACM Comput. Surv.* 54, 10s, Article 206 (Sept. 2022), 73 pages. doi:10.1145/3505243
- [41] Zhuoqun Yang, Zhi Li, Zhi Jin, and Yunchuan Chen. 2014. A Systematic Literature Review of Requirements Modeling and Analysis for Self-adaptive Systems. In *Requirements Engineering: Foundation for Software Quality*, Camille Salinesi and Inge van de Weerd (Eds.). Springer International Publishing, Cham, 55–71.
- [42] Mingqian Zheng, Jiaxin Pei, Lajanugen Logeswaran, Moontae Lee, and David Jurgen. 2024. When “A Helpful Assistant” Is Not Really Helpful: Personas in System

Prompts Do Not Improve Performances of Large Language Models. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen (Eds.). Association for Computational Linguistics, Miami, Florida, USA, 15126–15154. doi:10.18653/v1/2024.findings-emnlp.888

[43] Rui Zhu, Wenxin Li, and Canchang Jin. 2023. TAG: UML Activity Diagram Deeply Supervised Generation from Business Textual Specification. In *2023 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. 956–961. doi:10.1109/SANER56733.2023.00116